ALUNOS: Mateus Dassoler Spezia e Vinícius Oneda

ATIVIDADE 1

1) É possível garantir a exclusão mútua utilizando mecanismos como a instrução TSL (Test and Set Lock) ou desabilitando temporariamente as interrupções.

2) O principal problema é que um processo pode esquecer de reativar as interrupções, fazendo com que o sistema deixe de responder corretamente. Além disso, desabilitar interrupções impede o funcionamento adequado do sistema operacional e não é uma solução eficiente em sistemas com múltiplos processadores.

3) Condição de corrida é quando dois ou mais processos ou threads acessam e modificam um recurso compartilhado ao mesmo tempo, e o resultado final depende da ordem em que eles são executados. Ela pode ser evitada utilizando mecanismos de exclusão mútua.

4) Exclusão mútua é um mecanismo que impede que dois ou mais processos acessem simultaneamente uma região crítica ou um recurso compartilhado. Seu objetivo é evitar erros e inconsistências nos dados.

Exemplo: dois processos tentam atualizar o saldo de uma mesma conta bancária ao mesmo tempo. Com a exclusão mútua, apenas um processo realiza a atualização por vez, evitando que o saldo fique incorreto.

5) Seção crítica é a parte do código de um processo onde ocorre o acesso ou a modificação de um recurso compartilhado, como uma variável, arquivo ou banco de dados. Essa região deve ser executada por apenas um processo ou thread por vez.

ATIVIDADE 2

Process vs Threads: Processo é um programa em execução com seu próprio espaço de memória. Threads são unidades de execução dentro de um processo que compartilham seus recursos.

Communication: Shared Memory vs Messages: Na memória compartilhada, processos trocam informações utilizando uma área de memória comum. Em mensagens, eles se comunicam enviando e recebendo dados por meio do sistema operacional.

Shared Resources: Recursos que podem ser acessados por mais de um processo ou thread, como memória, arquivos, variáveis e dispositivos.

Mutual Exclusion and Synchronization: Exclusão mútua garante que apenas um processo ou thread acesse determinado recurso por vez. Sincronização coordena a execução para que os processos trabalhem na ordem adequada.

Race Conditions: Ocorrem quando dois ou mais processos ou threads acessam um recurso compartilhado simultaneamente e o resultado depende da ordem de execução.

Atomicity: É a propriedade de uma operação ser executada de forma indivisível, ou seja, completamente ou não ser executada.

Critical Section (Critical Region): Parte do código que acessa ou modifica um recurso compartilhado e, por isso, precisa ser protegida contra acessos simultâneos.

Mutex and Semaphores: Mutex é um mecanismo de exclusão mútua que permite que apenas uma thread acesse uma seção crítica por vez. Semáforos são mecanismos de sincronização que controlam o acesso a recursos compartilhados.

Busy Waiting: Situação em que um processo fica verificando repetidamente se um recurso está disponível, consumindo tempo de CPU enquanto espera.

Dining Philosophers: Problema clássico de sincronização que representa vários processos competindo por recursos compartilhados, podendo ocorrer espera indefinida ou deadlock.

Producer-Consumer Problem: Problema em que produtores geram dados e consumidores os utilizam, sendo necessário controlar o acesso a um buffer compartilhado para evitar conflitos.

Deadlock Problems: Ocorrem quando dois ou mais processos ficam bloqueados esperando por recursos que estão sendo utilizados uns pelos outros, fazendo com que nenhum consiga prosseguir.